Trabalho Prático 1 – Sistemas Operacionais – Threads e Controle de Concorrência

Beatriz Quaresma Athaide, Ruthelene Rodrigues Farias, Jonathas Monteiro da Trindade

[1]Instituto de Computação (IComp) – Universidade Federal do Amazonas (UFAM)
CEP 69080-900 – Manaus, AM – Brasil

{beatriz.quaresma,ruthelene.farias,jonathas.trindade}@icomp.ufam.edu.br

Abstract

This report presents implementations of two classical concurrent programming problems: readers–writers and producers–consumers. The programs were developed in C using POSIX threads and semaphores, with pseudo-random input, configurable delays, and event logs. For the readers–writers problem, a bank account was adopted as the shared resource, and three versions were developed: access without preference, allowing dirty reads; writer-priority access; and access without synchronization. For the producers–consumers problem, a circular buffer was used in versions with multiple producers and one consumer, multiple producers and consumers, and no concurrency control. The experiments show how synchronization prevents lost updates, inconsistent reads, buffer overwrites, empty-position reads, and duplicate consumption, while also illustrating blocking, possible reader starvation, and the effects of thread scheduling on execution order.

Resumo

Este relatório apresenta implementações de dois problemas clássicos de programação concorrente: leitores–escritores e produtores–consumidores. Os programas foram desenvolvidos em C com threads e semáforos POSIX, entrada pseudoaleatória, atrasos configuráveis e registro dos eventos de execução. No problema dos leitores e escritores, utilizou-se uma conta bancária como recurso compartilhado em três versões: acesso sem preferência, permitindo leitura suja; preferência de escritores; e acesso sem sincronização. No problema dos produtores e consumidores, empregou-se um buffer circular em versões com vários produtores e um consumidor, vários produtores e vários consumidores e ausência de controle de concorrência. Os experimentos mostram como a sincronização evita atualizações perdidas, leituras inconsistentes, sobrescritas no buffer, retiradas de posições vazias e consumos duplicados, além de evidenciarem bloqueios, compromissos de justiça e os efeitos do escalonamento sobre a ordem de execução das threads.

1 Introdução

Este trabalho aborda programação concorrente por meio de dois problemas clássicos de Sistemas Operacionais: leitores–escritores e produtores–consumidores. Em ambos os casos, várias threads acessam uma mesma estrutura de dados e precisam coordenar suas operações para

preservar a consistência do estado compartilhado. O objetivo é analisar como diferentes políticas de sincronização afetam a ordem de execução, o bloqueio das threads e o resultado das operações, além de demonstrar os erros que podem ocorrer quando o acesso concorrente não é controlado.

No problema dos leitores e escritores, adotou-se o tema de uma conta bancária: as threads leitoras consultam o saldo e as escritoras realizam depósitos ou saques. Foram desenvolvidas três versões. A primeira protege os escritores entre si, mas não estabelece preferência entre leitores e escritores, permitindo a ocorrência de leitura suja. A segunda concede preferência aos escritores e impede que leitores acessem a conta enquanto houver uma escrita ativa ou aguardando. A terceira remove os mecanismos de sincronização para evidenciar corridas entre escritores, como atualizações perdidas e divergências no saldo.

No problema dos produtores e consumidores, o recurso compartilhado é um buffer circular de tamanho fixo. A primeira versão possui vários produtores e um único consumidor; a segunda amplia a solução para vários produtores e vários consumidores; e a terceira elimina o controle de concorrência para expor sobrescritas, retiradas de posições vazias, consumos duplicados e inconsistências nos índices e contadores do buffer.

Todas as versões foram implementadas em C com a biblioteca POSIX Threads (`pthread`) e os mecanismos de sincronização disponibilizados por `semaphore.h`. A entrada é gerada pseudoaleatoriamente, com registro da semente para permitir a repetição das configurações, e atrasos artificiais são utilizados para tornar as diferentes intercalações mais visíveis. Os programas também registram a criação, o bloqueio, a entrada e a saída das regiões críticas e a finalização de cada thread, permitindo comparar nos logs o comportamento das versões sincronizadas com o das versões sem controle de concorrência.

2 Organização da Implementação

O tipo abstrato de dados da conta foi centralizado nos arquivos `conta.h` e `conta.c`, localizados diretamente no diretório do problema leitores–escritores, no mesmo nível dos diretórios correspondentes às três versões. Dessa forma, existe uma única implementação das operações sobre a conta, compartilhada por todos os programas, evitando duplicação de código e possíveis divergências entre versões.

O TAD `Conta` não possui sincronização interna. Essa decisão mantém separadas duas responsabilidades: o TAD implementa as operações bancárias e cada versão do programa define a política de concorrência exigida pelo experimento. Assim, uma mesma operação de depósito ou saque pode ser executada com proteção parcial, proteção completa ou sem controle, conforme a versão selecionada.

De forma análoga, o TAD `Buffer`, definido em `buffer.h` e `buffer.c` e localizado no diretório do problema produtores–consumidores, centraliza as operações sobre o vetor circular compartilhado (inserção, remoção e consulta de estado), também sem sincronização interna. A mesma separação de responsabilidades adotada para o TAD `Conta` se aplica aqui: o TAD implementa a estrutura de dados, e cada versão do programa define a política de concorrência correspondente.

Os arquivos `log.h` e `log.c` constituem um módulo auxiliar de apresentação. Eles não participam da lógica de sincronização, servindo apenas para padronizar a impressão de eventos como criação, bloqueio, entrada e saída da região crítica e finalização das threads. Essa separação facilita a leitura dos logs sem acoplar a formatação da saída aos algoritmos

concorrentes.

3 Versão 1: Acesso sem Preferência

3.1 Estratégia adotada

Na primeira versão, adotou-se uma estratégia deliberadamente assimétrica. Os escritores são sincronizados entre si por um semáforo binário denominado `mutex_escrita`, inicializado com o valor 1. Antes de executar um depósito ou saque, cada escritor realiza `sem_wait`; ao concluir a operação, realiza `sem_post`. Consequentemente, apenas um escritor pode modificar o saldo por vez. Caso outro escritor chegue durante essa operação, ele permanece bloqueado até a liberação do semáforo.

As threads leitoras, por outro lado, não aguardam esse semáforo antes de consultar o saldo. Portanto, a exclusão mútua é garantida na relação escritor–escritor, mas não na relação leitor–escritor. Essa escolha evita que dois escritores corrompam simultaneamente o saldo, ao mesmo tempo em que mantém intencionalmente a possibilidade de leitura suja solicitada pelo enunciado. A versão não pretende ser uma política adequada para uma aplicação bancária real; ela isola o fenômeno para fins didáticos.

Essa proteção parcial também diferencia a versão 1 da versão 3. Na versão 1, dois escritores nunca executam a região crítica simultaneamente. Na versão sem controle de concorrência, essa garantia deixa de existir e podem ocorrer atualizações perdidas entre escritores.

3.2 Ocorrência de leitura suja

Uma leitura suja pode ocorrer quando uma thread leitora consulta o saldo depois que um escritor iniciou sua operação, mas antes que ele a tenha concluído e saído da região crítica. Como o leitor não passa por `mutex_escrita`, nada impede essa sobreposição. O valor observado pelo leitor pertence, portanto, a um estado intermediário da execução e não a um estado já consolidado após a finalização do escritor.

Nos testes, leitores e escritores foram configurados com atraso de 3 segundos. O uso do mesmo atraso não é a causa da leitura suja: a causa é a ausência de sincronização entre leitores e escritores. Entretanto, tempos iguais tornam mais provável que as threads voltem a executar em instantes próximos, aumentando a sobreposição e fazendo o comportamento aparecer com mais clareza nos logs. Como o escalonamento é decidido pelo sistema operacional, a leitura suja é probabilística e pode não aparecer em todas as execuções.

3.3 Funcionamento do semáforo

Quando o semáforo está disponível, o primeiro escritor que executa `sem_wait` obtém acesso e entra na região crítica. Se outro escritor chegar antes de `sem_post`, sua chamada fica bloqueada na fila de espera do semáforo. Ao terminar, o escritor ativo executa `sem_post`, permitindo que uma das threads escritoras aguardando prossiga. Esse mecanismo corresponde às mensagens de escritor bloqueado apresentadas no log da aplicação.

A versão 1 utiliza apenas um semáforo porque existe somente uma relação a ser protegida: o acesso concorrente de escritor contra escritor. Incluir os leitores nesse mesmo bloqueio eliminaria justamente a leitura suja que a versão deve demonstrar.

4 Versão 2: Preferência de Escritores

4.1 Objetivo e estruturas de sincronização

A segunda versão implementa preferência de escritores. Quando existe um escritor ativo ou aguardando, novos leitores não podem iniciar consultas. Os leitores que já se encontram na região de leitura podem terminá-la, mas, após a chegada do primeiro escritor, a entrada de novos leitores é interrompida. Ao contrário da versão anterior, não ocorre leitura suja, pois leitores e escritores nunca utilizam o saldo simultaneamente.

Foram utilizados quatro semáforos, cada um com uma responsabilidade específica:

- `mutex_leitores`: protege o contador `readcount`, impedindo que incrementos ou decrementos concorrentes sejam perdidos;
- `mutex_escritores`: protege o contador `writecount`, que registra os escritores ativos ou aguardando;
- `r`: controla a admissão de novos leitores. O primeiro escritor que chega fecha essa entrada, e ela permanece fechada até a saída do último escritor;
- `w`: controla o acesso ao saldo, permitindo que ele seja usado por um grupo de leitores ou por um único escritor, mas nunca pelos dois grupos simultaneamente.

O leitor precisa passar por `r` antes de alterar `readcount`. Esse passo impede que um novo leitor ultrapasse um escritor que já está aguardando. O primeiro leitor do grupo bloqueia `w`, e o último o libera. Com isso, vários leitores podem consultar simultaneamente quando não há escritores, preservando a concorrência entre operações de leitura.

Do lado dos escritores, o primeiro escritor registrado bloqueia `r`, impedindo a entrada de novos leitores. Cada escritor utiliza `w` de forma exclusiva para atualizar a conta. Somente o último escritor libera novamente `r`. É essa combinação que produz a preferência observada: leitores que chegam durante uma sequência de escritas aguardam até que todos os escritores pendentes tenham terminado.

4.2 Justificativa para o uso de semáforos

Os semáforos são adequados ao algoritmo porque não possuem a regra de propriedade associada a `pthread_mutex`. Em um mutex POSIX, a thread que adquire o bloqueio deve ser a mesma que o libera. No algoritmo utilizado, o primeiro leitor pode bloquear `w` e o último leitor, possivelmente uma thread diferente, é responsável por liberá-lo. De modo semelhante, o primeiro escritor bloqueia `r` e o último escritor o libera. O par `sem_wait`/`sem_post` permite essa coordenação entre threads distintas.

Também foram preferidos semáforos bloqueantes a uma solução com espera ocupada. Enquanto aguarda, a thread é suspensa e não permanece consumindo tempo de processador para testar repetidamente uma condição.

4.3 Prevenção de deadlock e possibilidade de starvation

O emprego de múltiplos semáforos exige cuidado com deadlocks. Uma espera circular poderia surgir se diferentes threads adquirissem os recursos em ordens incompatíveis. Na implementação, leitores e escritores seguem uma ordem global e consistente de aquisição; não existe

um caminho em que uma thread retenha um recurso e aguarde outro que seja adquirido na ordem inversa por uma thread concorrente. Essa disciplina elimina a condição de espera circular necessária para a formação de deadlock.

Por outro lado, a preferência total de escritores introduz a possibilidade de *starvation* de leitores. Se escritores continuarem chegando sem intervalo, `writecount` não retorna a zero e `r` permanece bloqueado. Assim, leitores podem esperar por tempo indefinido. Esse é um custo conhecido da política escolhida, e não uma falha acidental da implementação.

4.4 Resultado observado

Em uma execução com quatro leitores, todos permaneceram bloqueados enquanto os escritores estavam ativos ou aguardando. Depois que o último escritor concluiu, os quatro leitores foram admitidos em conjunto e consultaram o mesmo saldo final:

$$1000 + 100 - 50 + 200 = 1250.$$

O resultado confirma a preferência dos escritores e a ausência de leitura suja. Nenhum leitor observou um valor intermediário; todos acessaram o estado consolidado após os três escritores. Embora o uso de atrasos ajude a tornar a ordem visível no terminal, a garantia de consistência decorre dos semáforos, e não dos tempos de espera configurados.

5 Portabilidade dos Semáforos POSIX

Leitores e escritores são threads de um único processo, de modo que semáforos anônimos seriam suficientes do ponto de vista do compartilhamento de memória. Entretanto, a família formada por `sem_init` e `sem_destroy` é marcada como obsoleta no macOS e não possui o mesmo suporte encontrado em sistemas Linux. Para evitar avisos e manter a execução no ambiente de desenvolvimento utilizado pela equipe, foram empregados semáforos POSIX nomeados, criados com `sem_open`.

Essa alteração não modifica a política de sincronização: tanto os semáforos anônimos quanto os nomeados oferecem as operações de espera e sinalização usadas pelo algoritmo. A diferença está no modo como o objeto é criado, identificado e removido. O caso evidencia que uma interface prevista pelo padrão POSIX pode apresentar níveis distintos de suporte entre sistemas operacionais.

6 Comparação entre as Versões

Na versão 1, apenas escritores são mutuamente exclusivos. A solução é simples e permite que leitores nunca aguardem por uma escrita, mas não garante que o valor lido corresponda a um estado consolidado da conta. Na versão 2, quatro semáforos coordenam os contadores, a entrada dos leitores e o acesso ao dado. Essa estrutura elimina leituras sujas e concede preferência a escritores, ao custo de maior complexidade e de tempos de espera potencialmente maiores para os leitores. Em síntese, a primeira versão demonstra as consequências de uma exclusão mútua incompleta, enquanto a segunda prioriza a consistência dos dados e explicita o compromisso entre segurança e justiça no escalonamento.

7 Geração da Entrada

A entrada dos programas é gerada aleatoriamente pelo módulo `gerador.h`/`gerador.c`, localizado no diretório comum, junto ao módulo de log. A cada execução são sorteados valores numéricos:

- **Leitores–escritores:** a quantidade de leitores, a quantidade de escritores, o saldo inicial da conta e o valor de cada escritor (positivo para depósito e negativo para saque);
- **Produtores–consumidores:** a quantidade de produtores, a quantidade de itens de cada produtor e a quantidade de consumidores.

A geração é encapsulada em três funções, declaradas em `gerador.h` e implementadas em `gerador.c`: `gerador_iniciar` recebe uma semente opcional (o valor 0 indica que uma semente deve ser derivada do horário atual combinado com o identificador do processo) e inicializa o gerador de números pseudoaleatórios da linguagem C; `gerador_leitores_escritores` e `gerador_produtores_consumidores` retornam, respectivamente, uma estrutura `ConfigLE` (quantidade de leitores e escritores, saldo inicial e o vetor de valores de cada escritor) e uma estrutura `ConfigPC` (quantidade de produtores, itens por produtor e quantidade de consumidores). Uma função auxiliar, `sortear_multiplo`, garante que valores como o saldo inicial e os valores de depósito ou saque sejam múltiplos de uma constante (`GER_PASSO`, igual a 10), apenas para facilitar a leitura dos logs.

As faixas de sorteio são definidas por constantes no arquivo `gerador.h`, de modo que outras configurações podem ser testadas sem alterar a lógica dos programas. Os valores são múltiplos de 10 apenas para facilitar a leitura dos logs. As faixas escolhidas para saldo inicial e saques impedem que o saldo fique negativo.

Cada execução imprime a semente utilizada no sorteio. Passando essa mesma semente como argumento na linha de comando, o programa gera a mesma entrada, o que permite repetir e documentar os exemplos deste relatório. Isso reproduz a *entrada*, mas não a saída linha por linha, pois a ordem em que o sistema operacional escalona as threads varia entre execuções. Além disso, a sequência produzida por `rand()` difere entre bibliotecas C, de modo que a mesma semente só reproduz a mesma entrada no mesmo sistema. O sorteio é feito pela thread principal, antes da criação das demais, pois `rand()` não é seguro para uso concorrente.

Os atrasos de cada tipo de thread, por sua vez, são constantes definidas no início de cada programa, o que permite testar qualquer configuração de atraso para qualquer tipo de thread.

8 Leitores–Escritores, Versão 3: Sem Controle de Concorrência

8.1 Estratégia adotada

A terceira versão não utiliza semáforos, mutex nem qualquer outra forma de espera: nenhuma thread aguarda outra. Seu objetivo é mostrar o problema do acesso simultâneo ao dado compartilhado. Como o enunciado prevê, o problema ocorre somente entre as threads escritoras, pois leitores apenas consultam o saldo.

A dificuldade de mostrar esse problema está em que a instrução `saldo += valor`, embora escrita em uma linha, corresponde a três passos: ler o saldo, calcular o novo valor e gravá-lo. Se dois escritores leem o mesmo saldo antes de qualquer um gravar, ambos calculam

a partir do mesmo valor antigo e o último a gravar apaga a atualização do outro. Esse fenômeno é chamado de *atualização perdida*. Em uma máquina real, a janela entre a leitura e a gravação dura apenas alguns nanossegundos, de modo que o erro apareceria de vez em quando. Para que ele fique visível em toda execução, o escritor executa os três passos separadamente e aguarda o atraso configurado entre a leitura e a gravação, simulando o tempo de processamento da operação. Esse é o mesmo atraso que, nas versões anteriores, é executado dentro da região crítica.

Como o TAD `Conta` oferece apenas depósito e saque, que executam os três passos de uma só vez, a gravação do resultado é feita diretamente no campo `saldo`, que é público na estrutura. Escritores e leitores são criados de forma alternada, com um pequeno intervalo entre as criações, para que leitores cheguem enquanto há escritores no meio da operação.

8.2 Instrumentação

O programa inclui verificações cujo único objetivo é tornar o erro visível no log; elas não corrigem nada. Antes de gravar, o escritor confere se o saldo mudou desde que ele o leu e, em caso afirmativo, registra que vai sobrescrever a atualização de outro escritor. Um contador atômico de escritores ativos permite ao leitor registrar que leu enquanto havia escritor em operação. No final, o programa compara o saldo esperado (saldo inicial mais a soma de todos os valores dos escritores) com o saldo obtido.

8.3 Exemplo de execução

A execução com a semente 106457680 sorteou cinco leitores, cinco escritores, saldo inicial 3220 e valores -170, -110, $+240$, -300 e $+440$, o que resulta em um saldo esperado de 3320. Os escritores 1 e 2 leram o mesmo saldo (3220). O escritor 1 gravou 3050 e, em seguida, o escritor 2, que continuava usando o valor lido anteriormente, avisou que o saldo havia mudado e gravou 3110, apagando a atualização do escritor 1:

```
[20:09:52] ESCRITOR 1 - LEU saldo = 3220 (vai aplicar -170)
[20:09:53] ESCRITOR 2 - LEU saldo = 3220 (vai aplicar -110)
[20:09:55] ESCRITOR 1 - GRAVOU saldo = 3050 (calculado a partir do saldo 3220 lido antes)
[20:09:56] ESCRITOR 2 - PROBLEMA: saldo mudou de 3220 para 3050 durante minha operacao; vou
    SOBRESCREVER a atualizacao de outro escritor
[20:09:56] ESCRITOR 2 - GRAVOU saldo = 3110 (calculado a partir do saldo 3220 lido antes)
...
Saldo esperado: 3320
Saldo obtido: 3550
```

O mesmo ocorreu com os demais escritores, e o saldo final foi 3550, diferente do esperado. Como o resultado depende da ordem em que cada thread lê e grava, o saldo final pode ficar acima ou abaixo do esperado; neste exemplo ficou 230 acima. Os leitores também consultaram valores que já estavam desatualizados, o que o programa sinaliza como possível leitura suja.

8.4 Comparação com a versão 1

Na versão 1, o semáforo `mutex_escrita` garante que dois escritores nunca executem a região crítica ao mesmo tempo, de modo que cada escritor lê o saldo já atualizado pelo anterior. Na versão 3 essa garantia deixa de existir, e a atualização perdida passa a ocorrer.

9 Produtores–Consumidores, Versão 1: Vários Produtores e Um Consumidor

9.1 Estratégia adotada

Nesta parte do trabalho, o recurso compartilhado é um buffer circular de tamanho fixo, implementado no TAD `Buffer` (`buffer.h` e `buffer.c`), seguindo a mesma separação de responsabilidades adotada para o TAD `Conta`: o buffer expõe apenas as operações de inserção, remoção e consulta de estado (cheio, vazio), sem qualquer sincronização interna. A política de concorrência é definida inteiramente pelo programa que utiliza o TAD.

A primeira versão do problema envolve várias threads produtoras e uma única thread consumidora. Cada produtor insere uma quantidade fixa de itens no buffer; o consumidor retira itens até que todos os produtores tenham terminado.

9.2 Funcionamento dos semáforos genéricos e do mutex

Foram utilizados dois semáforos genéricos e um mutex:

- `vazio`: inicializado com o valor igual à capacidade do buffer, representa o número de posições livres. Cada produtor executa `sem_wait(&vazio)` antes de inserir um item;
- `cheio`: inicializado com 0, representa o número de itens disponíveis para consumo. O consumidor executa `sem_wait(&cheio)` antes de retirar um item;
- `mutex`: semáforo binário (ou `pthread_mutex_t`) que garante exclusão mútua no acesso ao buffer, evitando que duas threads manipulem o vetor circular simultaneamente.

A ordem de aquisição é sempre a mesma: primeiro o semáforo genérico correspondente (`vazio` ou `cheio`), depois o `mutex`. Essa ordem é proposital: se o mutex fosse adquirido antes da verificação de vaga ou de item disponível, uma thread bloqueada manteria o mutex retido indefinidamente, impedindo que qualquer outra thread — inclusive aquela capaz de liberar a condição de espera — acessasse o buffer, o que caracterizaria uma situação de deadlock.

Ao final da inserção ou da remoção, o mutex é liberado antes da sinalização do semáforo genérico complementar (`sem_post(&cheio)` no produtor e `sem_post(&vazio)` no consumidor), completando o ciclo de sincronização.

9.3 Sinalização de encerramento

Diferentemente do problema de leitores e escritores, em que a quantidade de threads é conhecida antecipadamente por todas as partes, o consumidor de produtores-consumidores não tem, por si só, como saber quando não haverá mais itens a consumir. A solução adotada foi um valor sentinela: após o término de todas as threads produtoras (verificado por meio de `pthread_join`), a thread principal insere no buffer um valor reservado (`VALOR_SENTINELA = -1`), utilizando o mesmo protocolo de sincronização usado pelos produtores. Ao retirar esse valor, o consumidor interpreta-o como sinal de encerramento e finaliza sua execução.

9.4 Instrumentação dos bloqueios

Para tornar visível no log o momento em que uma thread é bloqueada — exigido pelo enunciado —, cada tentativa de acesso ao buffer é precedida por uma chamada não bloqueante (`sem_trywait`). Caso essa chamada falhe, a thread registra o evento de bloqueio (buffer cheio, para produtores, ou buffer vazio, para o consumidor) antes de então aguardar de fato pela chamada bloqueante `sem_wait`. Essa técnica não altera a semântica da sincronização; apenas permite diferenciar, no log, entre uma thread que obteve acesso imediato e uma que precisou esperar.

9.5 Exemplo de execução

Na execução com a semente 123, o gerador sorteou 3 produtores com 5 itens cada (15 itens no total) e 1 consumidor. Como os três produtores em conjunto produzem mais rápido do que o único consumidor consegue retirar, o buffer atinge a capacidade máxima repetidamente, bloqueando os produtores:

```
[16:29:04] PRODUTOR 1 - BLOQUEADA (buffer cheio, aguardando vaga)
[16:29:04] PRODUTOR 3 - BLOQUEADA (buffer cheio, aguardando vaga)
[16:29:04] CONSUMIDOR 1 - ENTROU na regiao critica
[16:29:04] CONSUMIDOR 1 - retirou o valor 201 do buffer
      buffer: [ 202 | _ | 301 | 102 | 302 ]
[16:29:04] CONSUMIDOR 1 - SAIU da regiao critica
```

Assim que o consumidor libera uma posição (`sem_post(&vazio)`), um dos produtores bloqueados é desbloqueado e prossegue. Como o escalonamento das threads é decidido pelo sistema operacional, repetir a execução com a mesma semente reproduz a mesma entrada (mesma quantidade de produtores e itens), mas não necessariamente a mesma sequência de bloqueios observada no log.

10 Produtores–Consumidores, Versão 2: Vários Produtores e Vários Consumidores

10.1 Diferenças em relação à Versão 1

A segunda versão mantém integralmente a lógica de sincronização da versão 1 — os mesmos semáforos genéricos `vazio` e `cheio` e o mesmo mutex —, alterando apenas a quantidade de threads consumidoras, que passa de uma para várias. Essa característica ilustra uma propriedade importante dos semáforos genéricos: como o contador pode ser incrementado e decrementado por threads distintas, a solução da versão 1 já era, por construção, correta para qualquer número de consumidores, sem exigir nenhuma alteração na lógica de exclusão mútua.

A única mudança estrutural adicional está na sinalização de encerramento: como agora existem várias threads consumidoras, um único valor sentinela não seria suficiente para encerrar todas elas. A thread principal insere, portanto, um valor sentinela para cada consumidor, garantindo que cada thread receba exatamente um sinal de encerramento e nenhuma permaneça bloqueada indefinidamente.

10.2 Comparação com a Versão 1

Com múltiplos consumidores, a taxa agregada de consumo tende a se aproximar ou superar a taxa de produção, o que reduz a frequência com que o buffer permanece cheio e aumenta a frequência com que ele permanece vazio, especialmente após o término dos produtores. Assim, a versão 2 tende a evidenciar com mais clareza o bloqueio de consumidores por buffer vazio, enquanto a versão 1, com um único consumidor mais lento que o conjunto de produtores, evidencia predominantemente o bloqueio de produtores por buffer cheio. Em ambos os casos, a exclusão mútua garantida pelo mutex impede que duas threads manipulem o vetor circular ao mesmo tempo, o que se confirma pela ausência de valores repetidos, sobrescritos ou perdidos nos logs — em contraste direto com o que se observa na versão 3, apresentada a seguir.

10.3 Exemplo de execução

Na execução com a semente 7, o gerador sorteou 2 produtores com 3 itens cada (6 itens no total) e 3 consumidores. Logo no início, os três consumidores já retiram itens mais rápido do que os dois produtores conseguem repor, e um consumidor é bloqueado com o buffer vazio:

```
[16:31:47] CONSUMIDOR 2 - ENTROU na regiao critica
[16:31:47] CONSUMIDOR 2 - retirou o valor 201 do buffer
      buffer: [ _ | _ | _ | _ | _ ]
[16:31:47] CONSUMIDOR 2 - SAIU da regiao critica
[16:31:47] CONSUMIDOR 3 - CRIADA
[16:31:47] CONSUMIDOR 3 - BLOQUEADA (buffer vazio, aguardando item)
[16:31:48] PRODUTOR 1 - ENTROU na regiao critica
[16:31:48] PRODUTOR 1 - inseriu o valor 102 no buffer
      buffer: [ _ | _ | 102 | _ | _ ]
[16:31:48] PRODUTOR 1 - SAIU da regiao critica
[16:31:48] CONSUMIDOR 3 - ENTROU na regiao critica
[16:31:48] CONSUMIDOR 3 - retirou o valor 102 do buffer
```

Em outras execuções, com uma proporção diferente entre produtores e consumidores ativos, é o bloqueio por buffer cheio que predomina, como na execução com a semente 123 apresentada na versão 1. A ocorrência de um ou outro cenário depende da relação entre a taxa de produção e a taxa de consumo em cada execução específica, e não é garantida em toda execução — assim como a leitura suja da versão 1 de leitores e escritores, discutida anteriormente.

11 Produtores–Consumidores, Versão 3: Sem Controle de Concorrência

11.1 Estratégia adotada

Nesta versão não há semáforos nem mutex: o produtor não aguarda vaga no buffer e o consumidor não aguarda item. Isso permite observar três tipos de problema:

- **Corrida nos índices:** dois produtores leem o mesmo índice de entrada e gravam na mesma posição, de modo que um valor apaga o outro. Da mesma forma, dois consumidores podem ler o mesmo índice de saída e consumir o mesmo valor duas vezes. O contador de itens do buffer também perde atualizações;
- **Overflow:** com o buffer cheio, o produtor continua e sobrescreve dados que ainda não foram consumidos;
- **Underflow:** com o buffer vazio, o consumidor continua e lê uma posição vazia, avançando o índice de saída e podendo deixar o contador de itens negativo.

Assim como na versão de leitores e escritores, a inserção e a retirada são executadas em passos separados (ler o índice, aguardar uma pausa curta e só então usá-lo), pois em hardware real essa janela é muito pequena e o erro apareceria raramente. As operações são escritas diretamente sobre os campos do buffer, sem as funções `buffer_inserir` e `buffer_retirar` do TAD, que impedem inserção em buffer cheio e retirada em buffer vazio e esconderiam o problema. Pelo mesmo motivo, o buffer é exibido pelo conteúdo real do vetor, e não pelo campo que registra a quantidade de itens, que é justamente um dos campos que se corrompem.

Sem semáforos não é possível usar com segurança o valor de encerramento das outras versões. Por isso, cada produtor produz uma quantidade fixa de itens e cada consumidor realiza uma quantidade fixa de retiradas, correspondente ao total de itens dividido entre os consumidores. O programa sempre termina, mesmo com o buffer corrompido.

11.2 Contabilidade final

Como os valores produzidos são únicos (o valor de cada item é formado pelo identificador do produtor e pelo número do item), o programa registra quantas vezes cada valor foi consumido. No final, apresenta os valores produzidos e nunca consumidos, o número de sobrescritas, o número de leituras de posições vazias e o número de consumos em duplicidade, além de comparar a quantidade registrada no buffer com o número de itens realmente presentes nas posições. Os contadores usam operações atômicas para que a própria contabilidade não seja uma fonte de erro.

11.3 Exemplo de execução

A execução com a semente 2049731252 sorteou dois produtores com cinco itens cada e dois consumidores. Logo no início, um consumidor retirou uma posição vazia e a quantidade registrada no buffer ficou negativa. Mais adiante, um valor foi sobrescrito por outro produtor antes de ser consumido e o valor 204 foi consumido por dois consumidores diferentes:

```
[20:11:32] CONSUMIDOR 2 - PROBLEMA: posicao 0 esta VAZIA; consumindo mesmo assim (buffer vazio
    )
      buffer: [ _ | _ | _ | _ | _ ] (entrada=1 saida=1 quantidade=-1)
[20:11:33] PRODUTOR 2 - PROBLEMA: posicao 1 ainda guarda o valor 102 (nao consumido);
    SOBRESCREVENDO com 202 - dado PERDIDO
[20:11:39] CONSUMIDOR 1 - PROBLEMA: o valor 204 JA foi consumido por outro consumidor (consumo
     em duplicidade)
```

O resumo final dessa execução foi:

```
Valores produzidos e NUNCA consumidos (sobrescritos ou esquecidos no buffer): 101 102 104 105
    203
Sobrescritas de dado nao consumido: 5
Leituras de posicao vazia (lixo): 4
Consumos em duplicidade: 1
```

Dos dez itens produzidos, cinco nunca foram consumidos: quatro foram sobrescritos por outro produtor e o valor 101 permaneceu esquecido no buffer. Quais problemas aparecem, e quantas vezes, varia a cada execução, pois depende do escalonamento das threads. As mensagens de sobrescrita registradas durante a execução indicam que uma posição ocupada foi reutilizada, mas apenas o resumo final, baseado na contagem dos valores, informa com certeza quais itens nunca foram consumidos: em alguns casos o valor sobrescrito já havia sido lido por um consumidor que ainda não havia liberado a posição.

11.4 Limitações

A corrida existe mesmo sem as pausas artificiais, mas com probabilidade muito baixa; as pausas servem apenas para torná-la visível. Além disso, a versão depende de acessos concorrentes sem sincronização, o que em C constitui comportamento indefinido; aqui isso é intencional, pois é exatamente o fenômeno que se deseja demonstrar.

12 Fontes Utilizadas nesta Parte

Esta seção foi elaborada a partir do enunciado do Trabalho Prático 1 da disciplina de Sistemas Operacionais, dos materiais apresentados em aula sobre threads, semáforos, leitores–escritores e deadlocks, do código e dos logs produzidos pela equipe e de conversa com assistente de inteligência artificial para apoio à organização e à revisão da explicação técnica.